ACM-ICPC Algorithm Template

目录

降维技巧
ARC098B Xor Sum 2

降维技巧

[L3] 降维技巧

一、单调性相关

双指针

- **双指针**：是一种使用两个指针来维护一段序列区间的方法，可以解决涉及“连续区间”的寻找或统计。
- **核心原理**：队列，右指针增加相当于入队，左指针增加相当于出队。
- **相关例题**：P1638 逛画展 | P3143 钻石收藏家

单调栈

- **单调栈**：是一种内部元素具有单调性的栈，可以解决求“以某个值为最值的最大区间”等问题。
- **核心原理**：单调性 + 栈
- **相关例题**：P2866 Bad Hair Day S | P1950 长方形

单调栈模板

> 找右边第一个比我大的 → 维护**单调递减**栈。

{% fold info @单调栈模版 %}

```cpp
// ans[i] 存储 i 右边第一个比 a[i] 大的元素下标
stack<int> stk;

for( int i = 1; i <= n; ++ i )
{
    // 第一个大的，反过来就是前面都是小的，所以是 <= 符号
    while( !stk.empty( ) && a[ stk.top( ) ] <= a[i] )
    {
        ans[ stk.top( ) ] = i;
        stk.pop( );
    }
    stk.push( i );
}
```

{% endfold %}

单调队列

- **单调队列**：是一种内部元素具有单调性的队列，可以解决求“区间内最值”的问题。
- **核心原理**：单调性 + 队列
- **相关例题**：P1886 滑动窗口 | P2216 理想的正方形

单调队列模板

> 求区间最大值 → 维护**单调递减**队列。

{% fold info @单调队列模版 %}

```cpp
deque<int> q;

for( int i = 1; i <= n; ++ i )
{
    // 1. 检查队头过期
    if( !q.empty( ) && q.front( ) <= i - K )
        q.pop_front( );

    // 2. 入队前元素处理
    while( !q.empty( ) && a[i] >= a[ q.back( ) ] )
        q.pop_back( );

    // 3. 入队 & 取值
    q.push_back( i );
    if( i >= K ) printf("%d ", a[ q.front( ) ] );
}
```

{% endfold %}

> 注：单调栈和单调队列一般不会单独作为考察点，往往在复杂问题中作为中间工具出现。

二、整体和部分

前缀和

- **前缀和**：可以在常数时间复杂度里 **查询区间和**。
- **核心公式**：$S_x = \sum_{i=1}^{x}(a_i) = a_1 + a_2 + \dots a_x$

差分

- **差分**：可以在常数时间复杂度里 **对序列进行区间操作**。
- **核心公式**：$D_x = a_x - a_{x-1}$

前缀和与差分

- **一维**：差分是前缀和的**逆运算**。
  - **推导**：将差分数组求前缀和，如下式：

$$\sum_{i=1}^{x}(D_i) = a_1 + \sum_{i=2}^{x}(a_x - a_{x-1}) = \sum_{i=1}^{x}(a_i) - \sum_{i=1}^{x-1}(a_i) = a_x$$

- **多维**：在多维空间中，利用容斥原理进行补偿。
  - 查询：$S[i][j] = S[i-1][j] + S[i][j-1] - S[i-1][j-1] + a[i][j]$。
  - 修改：通过四个顶点的修改来覆盖矩形区域。

离散化

> 离散，就是不连续的意思。

- **离散化**：在保持原数值之间相对大小关系不变的情况下，将大值域映射到紧凑的连续下标空间，是权值类数据结构（如树状数组、线段树）的预处理工具。
- **核心原理**：映射，只要相对顺序不变，对应的关系就不变。
- **相关例题**：
  - P1908 逆序对 BIT —— 权值数据结构预处理工具。
  - P3369 模板 普通平衡树 (权值线段树) —— 权值数据结构预处理工具。
  - P1955 程序自动分析 —— 离散化 + 并查集

离散化模板

{% fold info @离散化模版 - 静态数组版 %}

```cpp
const int maxn = 1e5 + 5;
int a[maxn];      // 原数组
int b[maxn];      // 离散化辅助数组
int n, m;         // n为实际元素个数，m为去重后元素个数

void discretize( )
{
    // 1. 拷贝原数组
    for( int i = 1; i <= n; ++ i ) b[i] = a[i];

    // 2. 排序
    sort( b + 1, b + n + 1 );

    // 3. 去重: unique返回去重后最后一个元素的下一个地址
    m = unique( b + 1, b + n + 1 ) - ( b + 1 );
}

// 4. 查询: 获取 x 离散化后的排名 (1-based)
int get_id( int x ) { return lower_bound( b + 1, b + m + 1, x ) - b; }
```

{% endfold %}

{% fold info @离散化模版 - **vector** 版 %}

```cpp
vector<int> nums; // 存储所有待离散化的值

void discretize( )
{
    sort( nums.begin( ), nums.end( ) );
    // erase + unique 是 STL 的标准去重全家桶
    nums.erase( unique( nums.begin( ), nums.end( ) ), nums.end( ) );
}

// 查询 x 映射后的值 (0-based)
int get(int x) { return lower_bound(nums.begin( ), nums.end( ), x) - nums.begin( ); }
```

{% endfold %}

{% fold info @离散化模版 - **map** 版 %}

```cpp
map< int, int > mp;
int cnt = 0;

int get_id( int x )
{
    if( mp.find( x ) == mp.end( ) ) mp[x] = ++ cnt;
    // 第一次见到 x，分配一个新 ID
    return mp[x];
}
```

{% endfold %}

ARC098B Xor Sum 2

ARC098B - Xor Sum 2

1. 核心逻辑

- **问题本质**: 统计满足“区间和等于区间异或和”的子区间数量。
- **破局转换**:
  1. **位运算性质**: $A + B = A \oplus B \iff (A\&B) = 0$。
     - 物理含义：二进制加法中没有任何一位发生进位。
     - 推论：区间 $[L, R]$ 合法 $\iff$ 该区间内所有数字在任意二进制位上至多只有一个 1。
  2. **单调性**: 若区间 $[L, R]$ 满足无进位加法，则其子区间必然也满足。
  3. **双指针滑动**: 固定 R，寻找最左侧的 L。当加入 $a[R]$ 导致按位与冲突时，右移 L 释放资源。

2. 代码实现

{% fold info @AcCode %}

```cpp
// AT_arc098_b Xor Sum 2
// Key Logic: Two Pointers + Bitwise AND for No-Carry Sum

#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const int maxn = 2e5 + 50;

int n;
ll a[ maxn ];

int main( )
{
    ios::sync_with_stdio( 0 );
    cin.tie( 0 );

    cin >> n;
    for( int i = 1; i <= n; ++ i ) cin >> a[i];

    ll ans = 0, cur_sum = 0;
    int L = 1;

    for( int R = 1; R <= n; ++ R )
    {
        // 若当前窗口已有位与 a[R] 冲突，则右移 L
        while( ( cur_sum & a[ R ] ) != 0 )
        {
            cur_sum ^= a[ L ];
            ++ L;
        }

        // 加入 a[R] 到窗口
        cur_sum |= a[ R ];

        // 贡献：以 R 为右端点的合法区间个数为 R - L + 1
        ans += ( ll )( R - L + 1 );
    }

    cout << ans << "\n";

    return 0;
}
```

{% endfold %}

3. 归档备注

- **状态维护**：使用 `cur_sum |= a[R]` 和 `cur_sum ^= a[L]` 动态维护窗口内的位状态，优于前缀和减法，且逻辑更贴合位运算本质。
- **相关知识点**：双指针